CS636 Project Report

Extending and Optimizing a GPU Multiversion B-link Tree

Based on the PACT 2022 MVGpuBTree implementation

Course: CS636
Authors: Himesh Pratap Singh(230478) and Arnav Gupta(230198)

April 26, 2026

Contents

Abstract **3**

1 Project Overview **3**

2 Background: B-trees and B-link Trees **3**

2.1 B-tree intuition . . . 3

2.2 B-link tree idea . . . 4

2.3 Why this matters on GPUs . . . 4

2.4 Short terminology guide . . . 4

3 Summary of the PACT 2022 Paper **4**

4 Repository and Implementation Overview **5**

4.1 Tile-cooperative node processing . . . 5

4.2 Node layout . . . 5

4.3 Allocators . . . 5

4.4 Clarification of non-obvious labels and notation . . . 6

5 Benchmarks Chosen for the Project **6**

6 Snapshot Handling and Versioning Logic **7**

7 Our Modifications **7**

7.1 Snapshot-counter extension . . . 7

7.2 Register-pressure reduction in the hot insert kernel . . . 7

8 Three Implementation States Evaluated **8**

9 Profiling Results **8**

9.1 NCU summary for the hot versioned insert kernel . . . 8

9.2 NCU summary for versioned and non-versioned find kernels . . . 9

9.3 Blink-only benchmark as a control case . . . 9

9.4 NSYS summary for `insert_find_bench` . . . 10

9.5 NSYS summary for `insert_find_blink_bench` . . . 10

9.6 Consolidated interpretation . . . 11

10 Why the Final Optimization Worked **11**

11 Why the Macro-Only Variant Matters **11**

12 Discussion of Runtime Costs **12**

13 Limitations **12**

14 Code and Reproducibility **12**

15 Contribution Split **14**

16 Conclusion **14**

A Important Repository Components **15**

Abstract

This project studies, modifies, and evaluates a concurrent GPU data structure based on the PACT 2022 paper *A GPU Multiversion B-Tree*. Our work focused on the MVGpuBTree implementation, which contains a GPU B-link tree and its multiversion extension. The project had two intertwined goals. The first was to understand how the paper's ideas are realized in CUDA code, especially the interaction between B-link traversal, version maintenance, and dynamic allocation. The second was to make targeted changes that improved correctness and performance in meaningful parts of the implementation.

The strongest optimization result obtained during the project was that the hot versioned insert kernel was reduced from **69 registers per thread** to **57 registers per thread** in the optimized version, which increased theoretical occupancy from **50.00%** to **66.67%**. An equally useful intermediate result was that changing only the broad inlining policy, without restructuring the insert logic, did **not** reduce the register count. This helped isolate the real source of the improvement and made the final conclusions much stronger.

1. Project Overview

Our project is based on the PACT 2022 MVGpuBTree implementation. The aim was not simply to profile an existing kernel, but to study, extend, and evaluate a dynamic concurrent GPU tree as a complete systems artifact. The codebase is interesting because it combines several difficult ingredients in one design: a B-tree-like search structure adapted for the GPU, B-link side links for concurrent traversal, multiversioning for snapshot-based reads, dynamic memory allocation on the GPU, and support for concurrent inserts, finds, range queries, and erases.

Because of this, the project required understanding the design proposed in the paper, the way that design is realized in CUDA, the performance bottlenecks that arise in concurrent tree updates, and the connection between source-level code structure and kernel resource usage. In practice, this meant working simultaneously at the algorithmic, implementation, and profiling levels rather than treating them as separate tasks.

2. Background: B-trees and B-link Trees

2.1 B-tree intuition

A B-tree is a balanced search tree in which each node stores multiple keys rather than a single key. Instead of descending through many binary nodes, traversal examines a compact array of keys inside one node and then selects the appropriate child pointer. This keeps the height of the tree relatively small and allows one node access to cover several key ranges at once. For GPU execution, that is valuable because each level of a pointer-based tree typically costs a memory access, and irregular memory access is already one of the main difficulties of GPU data structures.

2.2 B-link tree idea

A B-link tree extends a B-tree by adding two extra pieces of information to every node: a **high key**, which marks the upper bound of the node's key range, and a pointer to the node's **right sibling**. These additions are especially important under concurrency. If a traversal reaches a node and finds that the search key is no longer within that node's high-key bound, then it can move directly to the right sibling instead of restarting from the root. This local recovery mechanism makes concurrent structural changes, especially splits, much easier to tolerate.

2.3 Why this matters on GPUs

On GPUs, concurrent tree code is difficult because several hard problems appear at the same time. Traversal involves irregular memory accesses and pointer chasing, updates introduce synchronization and dynamic allocation, and different threads in the same warp or tile may not follow identical control flow. As a result, a concurrent GPU tree is challenging not only as a data-structure problem, but also as a GPU execution problem.

2.4 Short terminology guide

Since the reader may not already know the paper's terminology, it is useful to define a few recurring terms up front. A **branching factor** of `16` means that each node is designed around sixteen logical lanes. In this implementation, a **tile** is a cooperative group of `16` GPU threads that process one node together, so the branching factor and the tile width are intentionally matched.

A **high key** is the upper bound of the key range currently represented by a node. A **sibling pointer** is a pointer to the node immediately to the right, used when traversal needs to move laterally after a split. A **metadata lane** is a reserved lane in the node that stores structural information such as the high key, sibling pointer, lock bit, and leaf/internal-node bit. A **version lane** is another reserved lane that stores version-related information, specifically the node's snapshot number and a pointer to the next older version.

The update terms also deserve clarification. An **in-place update** means that the newest visible node is modified directly, while preserving older state only when required for snapshot correctness. An **out-of-place update** means that the updated logical state is materialized more explicitly in newly allocated storage, so the old state is preserved by construction. Finally, a **snapshot** is the logical version identifier used by readers to decide which historical node state is visible to them.

3. Summary of the PACT 2022 Paper

The paper proposes a GPU multiversion B-tree that supports concurrent operations and snapshot-based range queries. Its core idea is to build on top of a GPU B-link tree for safe concurrent structural traversal, and then add **multiversioning** so that readers can observe a logically consistent historical state while writers continue to update the newest state. In the repository, this appears as two related structures: a non-versioned `gpu_blink_tree` and a versioned `gpu_versioned_btree`.

The implementation supports two update strategies, namely **in-place versioned updates** and **out-of-place versioned updates**. In both cases, the versioned design relies on the same fundamental principle: a node carries version metadata, writers preserve older logical states when necessary, and readers use a snapshot timestamp to decide whether the current node is valid or whether an older version must be followed. This allows historical snapshot reads without forcing all readers and writers into a single serialized execution order.

4. Repository and Implementation Overview

The most important files for our work were `include/gpu_blink_tree.hpp`, `include/gpu_versioned_blink_tr` `include/node.hpp`, `include/versioned_node.hpp`, `include/btree_kernels.hpp`, `include/device_bump_al` `include/slab_alloc.hpp`, `include/memory_reclaimer.hpp`, `benchmarks/insert_find_bench.cu`, and `benchmarks/insert_find_blink_bench.cu`. These files together define the data structure, the node layout, the allocators, the kernels, and the benchmark drivers used in our evaluation.

4.1 Tile-cooperative node processing

The implementation uses branching factor $B = 16$, and this is not just a numerical constant. It determines how the data structure is mapped onto CUDA execution. One **tile of 16 GPU threads** is assigned to one logical node operation. The tile cooperatively loads a node, each thread corresponds to one lane in the node, and warp-level operations such as ballots and shuffles are used to search and manipulate node contents. This is the key GPU adaptation in the design: the node is treated as a small cooperative array rather than as an object processed independently by one thread.

4.2 Node layout

The versioned node layout is defined in `include/versioned_node.hpp`. For a node width of 16, fourteen lanes hold regular key-value data, one lane is reserved as the **version lane**, and one lane is reserved as the **metadata lane**. The metadata lane stores the high key, the sibling pointer, a lock bit, and a leaf/internal-node bit. The version lane stores a snapshot/version value together with a pointer to an older version. This compact packing is central to the design, because it preserves the tile-per-node execution model and strongly constrains how the implementation can be extended.

4.3 Allocators

The repository supports two allocators. The first is a **bump allocator**, implemented in `include/device_bump_allocator.hpp`, which allocates from a preallocated buffer using an atomic increment and is therefore very simple and fast. The second is a **slab allocator**, implemented in `include/slab_alloc.hpp`, which manages a preallocated pool in a more structured fixed-size fashion. Since inserts and splits create nodes dynamically, allocator behavior has a direct impact on runtime cost and is therefore an important part of the experimental story.

4.4 Clarification of non-obvious labels and notation

Some names that appear in the implementation and in the profiler outputs are not self-explanatory, so we clarify them here. Labels such as `bump 67108864` and `bump 33554432` refer to two template instantiations of the bump allocator with different preallocated capacities. Concretely, `67108864` means that the allocator is instantiated with space for 67,108,864 node objects, while `33554432` means space for 33,554,432 node objects. These numbers are allocator capacities, not profiler measurements.

Similarly, when the profiler prints long names such as `insert_in_place_kernel`, `insert_out_of_place_kernel` or `find_kernel`, these are the actual CUDA kernel entry points defined in `include/btree_kernels.hpp`. The words "in-place" and "out-of-place" describe the update strategy: in-place updates modify the newest visible node while preserving older state when necessary, whereas out-of-place updates build a new updated version more explicitly. In the same spirit, labels containing `slab` or `bump` identify the allocator used by that benchmarked tree type.

The allocator names themselves are also easy to misread if one is not already familiar with GPU dynamic allocation. The **bump allocator** is called "bump" because it allocates by bumping a single pointer or counter forward through a preallocated region. It is simple and fast, but it is not designed for sophisticated reuse of freed memory. The **slab allocator** is called "slab" because it manages memory in fixed-size allocation units taken from a preallocated pool. In this project, the allocator comparison matters because tree inserts and splits repeatedly allocate nodes, so allocator behavior directly affects insertion cost.

Finally, the occupancy-related terms in the profiler tables should be read carefully. "Registers per thread" is a compile-time resource metric for one kernel thread. "Theoretical occupancy" describes how many warps can reside on an SM given the kernel's resource usage, while "achieved occupancy" reflects what happened dynamically during execution. These two numbers do not always move together, especially for irregular data structures such as trees.

5. Benchmarks Chosen for the Project

Although the repository contains several benchmarks, our project focused on `insert_find_bench` and `insert_find_blink_bench`. The first benchmark is the most useful for studying the versioned tree because it compares versioned in-place insert, versioned out-of-place insert, non-versioned blink insert, and the corresponding find operations. This makes it possible to isolate the cost of versioning, the effect of allocator choice, and the difference between insert and find paths.

The second benchmark, `insert_find_blink_bench`, isolates the non-versioned blink tree. This is valuable because it provides a control case. If a change is truly specific to the versioned path, then the blink-only benchmark should remain structurally similar. We used this exact reasoning when interpreting the profiling results.

6. Snapshot Handling and Versioning Logic

One of the most important correctness ideas in the implementation is the snapshot mechanism. The versioned tree maintains a global snapshot counter, accessed through host-side and device-side `take_snapshot(...)` logic in `include/gpu_versioned_blink_tree.hpp`. In addition, each node's version lane stores the snapshot number of that node version together with the pointer or index of the next older version.

When a reader performs a snapshot-based lookup, it begins from the newest visible node, checks the node's version number, and if the node is too new for the requested snapshot, traverses backward through the version chain. This is the mechanism that allows the same tree to support concurrent updates and historical reads. It is also subtle, because it works correctly only if snapshot values remain valid, version pointers are not corrupted, and older versions remain available for as long as readers may still need them.

7. Our Modifications

Our work had two main categories: correctness/scalability changes and performance changes.

7.1 Snapshot-counter extension

The repository README notes that snapshots are limited to a maximum of $2^{32} - 1$ and can be extended further. Based on this and on the packed node layout, we changed the global backing counter so that it uses a 64-bit type for the **counter mechanism**. The important design choice here was that the **global counter backing store** was widened while the **node layout** remained compact. This preserved the packed lane structure instead of forcing a much larger redesign of the version lane.

The purpose of this change was to avoid silent wraparound of the global snapshot counter, make the implementation safer for long-running workloads, and preserve the intended semantics of snapshot-based lookup. This change is therefore best viewed as a **correctness and scalability** improvement rather than a direct throughput optimization.

7.2 Register-pressure reduction in the hot insert kernel

The main performance investigation centered on the versioned `insert_in_place_kernel`. Baseline profiling showed that this kernel had high register usage and reduced theoretical occupancy, making it a strong candidate for optimization. To address this, we explored changes to inlining policy, function boundaries, and kernel structure.

The final optimized state included a noinline-capable device macro, heavier node methods that were allowed to compile as separate functions, extracted split-handling helpers, and launch-bounds guidance for the insert kernels. Together, these changes produced the strongest improvement in the project.

8. Three Implementation States Evaluated

One useful aspect of the project is that we were able to observe three distinct implementation states. The first was the original or vanilla implementation, which served as the baseline. The second was a macro-only noinline variant, in which the original in-place insert logic was kept structurally intact but the broad inlining policy was changed. The third was the final optimized variant, in which the hot insert implementation was structurally refactored in addition to the inlining changes.

This three-state evaluation was important because it allowed us to isolate what actually caused the improvement, rather than only comparing a baseline and a final state.

9. Profiling Results

This section presents the profiling results in a compact analysis-oriented format. We deliberately include both the most relevant and some secondary metrics, because in a GPU data-structure workload the "irrelevant" numbers often help explain why the main numbers behave the way they do. The key benchmarks are `insert_find_bench` and `insert_find_blink_bench`, and the key implementation states are the original baseline, the macro-only noinline variant, and the final optimized variant.

9.1 NCU summary for the hot versioned insert kernel

The main kernel of interest is the versioned `insert_in_place_kernel` from `insert_find_bench`. Its NCU statistics across the three implementation states are shown below.

Table 1: NCU summary for the hot versioned insert kernel

Metric	**Original**	**Macro-only**	**Final optimized**
Registers per thread	69	69	57
Theoretical occupancy	50.00%	50.00%	66.67%
Achieved occupancy	47.26%	15.36%	15.21%
Register-limited residency	3 blocks/SM	3 blocks/SM	4 blocks/SM

The most important observation is that the register count does **not** change between the original implementation and the macro-only noinline variant. The change appears only in the final optimized version, where the register count drops from `69` to `57`. This is strong evidence that the decisive factor was not the macro-level inlining change by itself, but the structural refactoring of the insert logic.

The achieved occupancy number is lower in the later runs than in the original baseline. This should not be interpreted as a contradiction. Theoretical occupancy is the cleaner indicator of how the kernel's compiled resource footprint changed, whereas achieved occupancy also reflects runtime behavior such as memory stalls, synchronization, and workload scale. For our project, the most defensible interpretation is that the final optimized kernel is structurally lighter in register usage even though runtime behavior is still dominated by irregular tree-insert costs.

9.2 NCU summary for versioned and non-versioned find kernels

To understand whether the optimization was localized to insert or broadly affected all tree operations, it is useful to compare the `find_kernel` results as well.

Table 2: NCU summary for representative find kernels

Kernel / State	Regs/thread	Theor. Occ.	Achieved Occ.
Versioned `find_kernel` (baseline slab)	28	100.00%	96.35%
Versioned `find_kernel` (baseline bump)	29	100.00%	96.39%
Versioned `find_kernel` (macro-only slab)	45	66.67%	32.12%
Versioned `find_kernel` (macro-only bump)	46	66.67%	32.75%
Versioned `find_kernel` (optimized slab)	31	100.00%	32.57%
Versioned `find_kernel` (optimized bump)	31	100.00%	32.65%
Blink `find_kernel` (baseline slab)	22	100.00%	96.29%
Blink `find_kernel` (baseline bump)	22	100.00%	96.30%
Blink `find_kernel` (macro-only slab)	36	100.00%	32.51%
Blink `find_kernel` (macro-only bump)	37	100.00%	31.61%
Blink `find_kernel` (latest blink-only slab)	36	100.00%	32.56%
Blink `find_kernel` (latest blink-only bump)	37	100.00%	32.13%

These numbers are useful even though find is not the main optimization target. They show that insert and find are affected differently by code-generation changes. In particular, the strongest structural gain is still concentrated in the hot versioned insert kernel. The find path is useful as supporting evidence because it shows that the overall tree code changed in more complicated ways than a single metric can capture, but the main targeted improvement is still clearly the insert-side register reduction in the final optimized version.

9.3 Blink-only benchmark as a control case

The blink-only benchmark, `insert_find_blink_bench`, provides a clean control case because it removes multiversioning and preserves only the non-versioned B-link tree logic.

Table 3: Blink-only NCU comparison

Metric	Baseline Slab	Macro-only Slab	Baseline Bump	Macro-on
Insert regs/thread	57	62	47	62
Insert theor. occ.	66.67%	66.67%	66.67%	66.6
Insert achieved occ.	62.63%	30.63%	63.78%	29.6
Find regs/thread	22	36	22	37
Find theor. occ.	100.00%	100.00%	100.00%	100.0
Find achieved occ.	96.29%	32.56%	96.30%	32.1

Some of these values are not central to the main claim, but they are still informative. The blink-only benchmark shows that structural and compiler-facing changes can alter kernel resource usage even outside the versioned insert path. However, the key result remains that the **specific** improvement from `69` to `57` registers/thread appears only in the final optimized version of the

versioned insert kernel. The blink-only benchmark therefore helps separate local effects from global ones and makes the final interpretation more credible.

9.4 NSYS summary for `insert_find_bench`

The NCU data explains how the kernels are compiled; the NSYS data helps explain where time goes during execution. The table below summarizes the most relevant kernel-level times from the collected runs.

Table 4: NSYS summary for `insert_find_bench`

Metric	Baseline	Macro-only / Current	Final Optimized
`cudaMalloc` API share	62.6%	50.9%	51.6%
Versioned out-of-place insert (slab)	242.91 ms	1.293 ms	687.24 us
Versioned out-of-place insert (bump)	233.71 ms	1.099 ms	663.98 us
Versioned in-place insert (slab)	190.17 ms	877.93 us	514.95 us
Versioned in-place insert (bump)	326.65 ms	838.60 us	554.98 us
Blink insert (slab)	230.35 ms	957.83 us	270.12 us
Blink insert (bump)	187.76 ms	902.47 us	254.85 us
Versioned find (slab avg)	10.97 ms	89.34 us	39.44 us
Versioned find (bump avg)	10.96 ms	79.87 us	38.75 us
Blink find (slab)	823.45 us	72.90 us	23.90 us
Blink find (bump)	822.62 us	61.38 us	23.10 us

These timing values should be interpreted more carefully than the NCU register results, because absolute runtime numbers are more sensitive to workload size, replay conditions, and other experimental details. Even so, several broad patterns are stable. First, insert remains much heavier than find in every state. Second, allocator and versioning overhead continue to matter even after structural optimization. Third, `cudaMalloc` continues to occupy a substantial portion of API time, which reinforces the point that dynamic GPU data structures are shaped not just by kernel arithmetic, but by allocation and runtime overhead as well.

9.5 NSYS summary for `insert_find_blink_bench`

The blink-only benchmark shows a similar asymmetry between insert and find.

Table 5: NSYS summary for `insert_find_blink_bench`

Metric	Baseline	Macro-only / Current	Final Optimized
`cudaMalloc` API share	74.0%	12.9%	12.7%
Blink insert (slab)	279.93 ms	1.006 ms	271.78 us
Blink insert (bump 67108864)	222.81 ms	917.41 us	265.25 us
Blink insert (bump 33554432)	151.06 ms	867.78 us	252.96 us
Blink find (slab)	823.71 us	72.48 us	23.97 us
Blink find (bump 67108864)	822.97 us	62.30 us	23.04 us
Blink find (bump 33554432)	821.82 us	61.89 us	23.01 us

Again, these data are useful mainly for pattern recognition rather than for making a single headline claim. The clearest pattern is that insert remains the dominant operation, while find

stays comparatively light. The blink-only data also confirms that even without multiversion logic, a dynamic GPU tree is still controlled by structural costs such as allocation, memory access, and traversal complexity.

9.6 Consolidated interpretation

Taken together, the NCU and NSYS results lead to a consistent interpretation. The final optimized version produced a genuine structural improvement in the hot versioned insert kernel, reducing its register count from `69` to `57` and moving it from a `50.00%` theoretical-occupancy regime to a `66.67%` regime. The macro-only noinline variant did not achieve this, which means the improvement cannot be attributed to inlining policy alone.

At the same time, the results also show that the tree remains a difficult GPU workload. Insert continues to dominate runtime, allocator overhead remains visible, and achieved occupancy does not automatically track theoretical occupancy. This is exactly the kind of behavior one would expect from a concurrent, pointer-based, dynamically allocating GPU data structure. For that reason, the most reliable headline result of the project is not a universal speedup claim, but the more precise statement that the final structural refactor reduced register pressure in the hot versioned insert kernel in a way that simpler changes did not.

10. Why the Final Optimization Worked

The optimized version reduced register pressure because it improved the structure of the compiled hot path. In the original versioned in-place insert logic, traversal state, split logic, node-copy bookkeeping, and multiple helper operations were all visible together inside a large compiled body. As a result, too many intermediate values remained live at the same time.

This matters because if many values are live simultaneously, each thread needs more registers. Since each SM has a finite register budget, high register usage reduces how many blocks and warps can remain resident, and that in turn lowers theoretical occupancy.

The structural refactor helped because it reduced how much heavy logic was merged into one giant body. By breaking out expensive sections and changing the compiler's visibility of function boundaries, the optimized version reduced live-variable pressure in the hot insert path. This is exactly what is reflected in the observed change from 69 to 57 registers per thread.

11. Why the Macro-Only Variant Matters

The macro-only noinline experiment was important because it gave us a negative result that is still technically meaningful. It showed that changing the macro alone did not reduce the hot insert kernel's register count, that the original structural complexity of the insert function still dominated code generation, and that the final improvement therefore depended on more than just a broad noinline policy.

This strengthens the causal argument behind the final optimized result. In many optimization studies, such intermediate negative results are not emphasized, but here they were useful because they made the final conclusion much more defensible.

12. Discussion of Runtime Costs

The measured runtime behavior reflects several overlapping costs: tree traversal cost, insert structural work, version maintenance, allocator overhead, and memory-system latency. Dynamic trees are hard on GPUs because they combine pointer chasing, branch divergence, synchronization, irregular memory access, and dynamic allocation. These factors interact with one another, which is why tree kernels are much harder to optimize than regular array-based GPU kernels.

Even though the repository uses bump and slab allocators instead of naive per-node `cudaMalloc`, allocation-related cost remains visible in runtime profiles. That is expected because insert-heavy workloads still depend on rapid node creation during splits and version copying. Find is lighter because it mostly traverses nodes, compares keys, optionally follows version pointers, and returns a result; it does not usually perform structural modification, splitting, or new allocation.

13. Limitations

The strongest and safest claim from the project is about **register and occupancy improvement**, not a universal speedup claim for every benchmark. Insert remains the dominant cost even after optimization, snapshot extension was a correctness/scalability improvement rather than a direct throughput optimization, and the blink-only benchmark mainly serves as a control case rather than as the primary location of the optimization result. These limitations do not weaken the work; they make the conclusions more precise.

14. Code and Reproducibility

The original implementation studied in this project is the Owens Group repository released with the PACT 2022 work: https://github.com/owensgroup/MVGpuBTree. Our modified implementation, containing the snapshot-counter change, the macro-only variant explored during the project, and the final optimized insert-path version, is maintained at: https://github.com/arnavg11/modified_MVGpuBT

All profiling should be performed on the GPU cluster rather than on a local CPU-only development machine. In our case, the important benchmark targets were `insert_find_bench` and `insert_find_blink_bench`, and profiling was carried out after explicitly configuring CUDA, selecting the correct architecture, and disabling CUDA MPS during benchmark execution. A typical build sequence on the cluster is shown below.

```
cd /users/misc/arnavgupta23/MVGpuBTree
rm -rf build
mkdir build
cd build

cmake .. \
  -Dbuild_tests=OFF \
  -Dbuild_benchmarks=ON \
  -DCMAKE_CXX_COMPILER=/usr/bin/c++ \
  -DCMAKE_CUDA_COMPILER=/usr/local/cuda/bin/nvcc \
  -DCMAKE_CUDA_HOST_COMPILER=/usr/bin/c++ \
  -DCMAKE_CUDA_ARCHITECTURES=86 \
```

```
  -DCMAKE_CUDA_RUNTIME_LIBRARY=Shared \
  -DCMAKE_TRY_COMPILE_TARGET_TYPE=STATIC_LIBRARY \
  -DCMAKE_CUDA_STANDARD_COMPUTED_DEFAULT=14 \
  -DCMAKE_CUDA_EXTENSIONS_COMPUTED_DEFAULT=ON \
  -DCMAKE_CUDA_COMPILER_FORCED=ON \
  -DCMAKE_CUDA_COMPILER_WORKS=ON \
  -DCUDA_ARCHS="--generate-code=arch=compute_86,code=sm_86;"

cmake --build . --target insert_find_bench insert_find_blink_bench -- -
    j1
```

Once the binaries have been built on the cluster, the following commands can be used to profile both benchmarks with Nsight Compute and Nsight Systems. The output is written to a `profile_txt` directory in the repository root, while temporary `nsys` report files are placed in a user-specific temporary directory.

```
cd /users/misc/arnavgupta23/MVGpuBTree/build

mkdir -p ../profile_txt
mkdir -p /tmp/nsight_$USER
mkdir -p /tmp/mps_disabled

export CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_disabled
unset CUDA_MPS_LOG_DIRECTORY
unset CUDA_MPS_ACTIVE_THREAD_PERCENTAGE
export TMPDIR=/tmp/nsight_$USER

for exe in insert_find_bench insert_find_blink_bench; do
  ncu --target-processes all \
    --replay-mode application \
    --kernel-name regex:".*(insert|find).*kernel.*" \
    --launch-count 20 \
    --section LaunchStats \
    --section Occupancy \
    ./bin/$exe \
    --num-keys=1000 \
    --num-experiments=1 \
    --exist-ratio=1.0 \
    --validate=false \
    > ../profile_txt/${exe}_ncu.txt 2>&1

  nsys profile \
    --force-overwrite true \
    --trace=cuda,nvtx,osrt \
    --sample=none \
    --cpuctxsw=none \
    -o /tmp/nsight_$USER/${exe}_nsys \
    ./bin/$exe \
    --num-keys=1000 \
    --num-experiments=1 \
    --exist-ratio=1.0 \
    --validate=false \
```

```
    > ../profile_txt/${exe}_nsys_profile.txt 2>&1

  nsys stats \
    --report cuda_api_sum,cuda_gpu_kern_sum,cuda_gpu_mem_time_sum \
    --format csv \
    /tmp/nsight_$USER/${exe}_nsys.nsys-rep \
    > ../profile_txt/${exe}_nsys.txt 2>&1
done
```

After profiling on the cluster, the generated text reports can be copied back to a local machine using `scp`. For example, the following command copies the entire `profile_txt` directory to a local `Downloads` folder:

```
scp -r arnavgupta23@gpu3.cse.iitk.ac.in:/users/misc/arnavgupta23/
   MVGpuBTree/profile_txt \
  /Users/agupta/Downloads/
```

15. Contribution Split

This project was completed jointly by us, and we record the effort split as **50% − 50%**. The work included reading and understanding the PACT 2022 paper, understanding the MVGpuB-Tree codebase, examining versioned-node and kernel logic, investigating snapshot handling, implementing and testing code changes, comparing baseline and modified profiler outputs, and documenting the implementation and results.

We consider the split even because both implementation understanding and evaluation were central to the project outcome, and the project depended equally on design comprehension, code modification, and result interpretation.

16. Conclusion

This project successfully studied and optimized a non-trivial GPU data structure implementation. Our work involved understanding a concurrent GPU B-link tree, reasoning about multiversion snapshot semantics, making correctness-oriented snapshot changes, performing targeted performance optimization on the hot insert kernel, and evaluating three implementation states to isolate the real source of improvement.

The strongest technical result is that, in the final optimized version, the hot versioned insert kernel was reduced from **69** to **57 registers per thread**, increasing theoretical occupancy from **50.00%** to **66.67%**. Equally important is the methodological conclusion that this improvement did not come from a simple macro-only noinline change. It required structural changes to the hot insert implementation. Taken together, these results show that the project achieved both a meaningful implementation-level improvement and a clear understanding of why that improvement occurred.

A. Important Repository Components

The most important repository components for this project were `include/gpu_blink_tree.hpp`, which contains the non-versioned B-link tree; `include/gpu_versioned_blink_tree.hpp`, which contains the versioned tree and the main insert/find logic; `include/versioned_node.hpp`, which defines the versioned node layout, version metadata, splitting, and locking; `include/btree_kernels.hpp`, which provides the CUDA kernels; `include/device_bump_allocator.hpp` and `include/slab_alloc.hpp`, which provide the two allocator families; `include/memory_reclaimer.hpp`, which contains reclamation logic; and the benchmark drivers `benchmarks/insert_find_bench.cu` and `benchmarks/insert_find_b`